

JWT-Based Blog Management REST API

Updated Product Requirements and Codex Implementation Specification

Innovation Within Learning Project

17 July 2026

Contents

1. Document Purpose	4
2. Mandatory Instructor Requirements	4
2.1 Class-based implementation	4
2.2 UUID user identifiers	4
2.3 All request checks must be performed in middleware	4
2.4 Controllers must contain no business or checking logic	5
2.5 Constants must be stored in the correct constants folder	5
2.6 Git branching and proper commits are mandatory	5
2.7 Prettier formatting is mandatory	6
3. Product Overview	6
4. Problem Statement	6
5. Project Objectives	6
6. Technology Stack	7
6.1 Core technologies	7
6.2 Runtime dependencies	7
6.3 Development dependencies	7
6.4 Package responsibilities	7
7. Target Users	7
7.1 Guest user	7
7.2 Registered user	7
8. Product Scope	8
8.1 In scope	8
8.2 Out of scope	8
9. Core Architecture Principles	8
9.1 Class-based modular architecture	8
9.2 Middleware-first request processing	9
9.3 Thin controllers	9
9.4 Services execute use cases	9
9.5 Database services only access the database	9
9.6 Query files contain SQL only	10
9.7 Constants eliminate magic values	10
10. Functional Requirements	10
10.1 User registration	10
Route	10

Request body	10
Middleware chain	10
Service processing	10
Expected response	10
10.2 User login	11
Route	11
Middleware chain	11
Controller and service	11
Expected response	11
10.3 Get current user	11
Route	11
Middleware chain	11
10.4 Create a blog post	11
Route	11
Middleware chain	12
10.5 Get all blog posts	12
Route	12
10.6 Get one blog post	12
Route	12
Middleware chain	12
10.7 Update a blog post	12
Route	12
Middleware chain	12
10.8 Delete a blog post	12
Route	12
Middleware chain	13
11. API Endpoints	13
11.1 Health	13
11.2 Authentication	13
11.3 Blogs	13
11.4 Optional user module after core completion	13
12. Database Requirements	13
12.1 Database	13
12.2 Users table with UUID primary key	14
12.3 Blogs table	14
12.4 Relationship	14
13. Required Project Structure	14
14. Required Request Flow	16
14.1 Generic flow	16
14.2 Update-blog flow	16
15. Class-Based Coding Standard	17
15.1 Controller class	17
15.2 Middleware class	17
15.3 Service class	17
15.4 Database service class	18
16. File Responsibilities	18
16.1 Routes	18
16.2 Middleware	19
16.3 Controllers	19
16.4 Main services	19
16.5 Database services	19
16.6 Query files	19

16.7 Filter services	19
17. Constants Requirements	19
17.1 Auth constants example	19
17.2 Blog constants example	20
17.3 Constants placement rules	20
18. Authentication and Security Requirements	20
18.1 Passwords	20
18.2 JWT payload	20
18.3 JWT expiry	20
18.4 Ownership	21
18.5 SQL injection prevention	21
19. Validation and Middleware Matrix	21
20. Error Handling Requirements	21
20.1 Required status codes	21
20.2 Error middleware order	22
21. Environment Configuration	22
21.1 .env	22
21.2 .env.example	22
22. Prettier and Formatting Requirements	22
22.1 .prettierrc.json	22
22.2 .prettierignore	22
22.3 Package scripts	22
22.4 Formatting gate	23
23. Postman Testing Plan	23
23.1 Collection structure	23
23.2 Environment variables	23
23.3 Login test script	23
23.4 Required tests	24
Authentication	24
Blogs	24
UUID-specific tests	24
24. Git and GitHub Workflow	24
24.1 Required branches	24
24.2 Required workflow	25
24.3 Commit rules	25
24.4 Conventional commit examples	25
24.5 Suggested feature order	25
25. Development Milestones	26
Milestone 1: Project foundation	26
Milestone 2: MySQL and schema	26
Milestone 3: Registration	26
Milestone 4: Login and JWT	26
Milestone 5: Protected profile	26
Milestone 6: Blog creation and reading	26
Milestone 7: Blog update and deletion	27
Milestone 8: Errors and consistency	27
Milestone 9: Testing	27
Milestone 10: Documentation and release	27
26. Definition of Done	27

27. Acceptance Criteria	28
27.1 Architecture acceptance	28
27.2 Registration acceptance	28
27.3 Login acceptance	28
27.4 Blog acceptance	28
27.5 Git and formatting acceptance	28
28. Codex Execution Instructions	28
29. Future Enhancements	29
30. Final Product Summary	29

1. Document Purpose

This document is the updated product requirements document and implementation specification for a JWT-based Blog Management REST API. It is intended to be used both as a development guide and as the primary instruction file for Codex.

The application will be a backend-only REST API built with JavaScript, Node.js, Express.js, and MySQL. Registered users will be able to authenticate with JSON Web Tokens and manage their own blog posts.

This revision replaces the earlier functional-style design with a mandatory class-based architecture and adds stricter rules for middleware, UUID user identifiers, constants, Git workflow, commit quality, and Prettier formatting.

2. Mandatory Instructor Requirements

The following requirements are non-negotiable and take priority over all optional design choices.

2.1 Class-based implementation

- The application must use JavaScript classes instead of standalone functional controllers, services, middleware modules, or database services.
- Controllers, services, database services, helper services, filter services, and module-specific middleware must be implemented as classes.
- Dependencies should be passed through constructors where practical.
- Methods used as Express handlers must be bound to their class instance or defined so that **this** is preserved.
- Codex must not replace the class-based architecture with a functional architecture for convenience.

2.2 UUID user identifiers

- Every user must have a UUID as the primary identifier.
- User IDs must not use MySQL `AUTO_INCREMENT`.
- UUIDs should be generated in the application using Node.js `crypto.randomUUID()` before insertion.
- The MySQL learning-project representation should be `CHAR(36)`.
- The user UUID must be used in JWT payloads, foreign keys, request context, and API responses.
- Any route accepting a user ID must validate that the value is a valid UUID in middleware.

2.3 All request checks must be performed in middleware

All request-related checking must finish before the controller executes. This includes:

- Request-body validation
- Route-parameter validation
- Query-parameter validation
- Data normalization
- JWT presence and verification
- Logged-in user loading

- Duplicate email detection
- Login credential verification
- Resource existence checks
- Blog ownership checks
- Empty update detection
- Update-field allow-list checking
- Permission and authorization checking

Middleware must attach verified and prepared data to the request, for example:

```
req.validatedBody;  
req.auth;  
req.currentUser;  
req.blog;  
req.targetUser;
```

2.4 Controllers must contain no business or checking logic

Controllers are transport adapters only. A controller may:

- Read data already prepared by middleware
- Call one service method
- Return an HTTP response
- Pass an unexpected error to the centralized error handler when an async wrapper is not used

Controllers must not:

- Validate input
- Normalize input
- Compare passwords
- Verify tokens
- Query whether a record exists
- Check duplicate email
- Check ownership
- Decide permissions
- Build SQL
- Contain `if` statements for request checks
- Contain business rules

2.5 Constants must be stored in the correct constants folder

Fixed values must not be scattered as magic numbers or repeated strings.

Examples:

- `SALT_ROUNDS` belongs in `src/modules/auth/constant/auth.constant.js`.
- Password length rules belong in the auth constants file.
- Blog title length belongs in the blog constants file.
- Shared HTTP status codes belong in a global constants file.
- Shared application messages may be global; module-specific messages must stay inside the relevant module.
- Secrets and machine-specific values belong in `.env`, not in constants files.

2.6 Git branching and proper commits are mandatory

- Development must use feature branches.
- Direct feature development on `main` is not allowed.
- Commits must be small, focused, meaningful, and ordered.
- Commit messages must follow a consistent conventional style.
- Formatting-only changes must not be mixed with unrelated feature changes.
- Each branch must be formatted and manually tested before merge.

2.7 Prettier formatting is mandatory

- Prettier must be installed as a development dependency.
- A shared Prettier configuration must be committed.
- `npm run format` and `npm run format:check` scripts must be available.
- `npm run format:check` must pass before a branch is merged.

3. Product Overview

The JWT-Based Blog Management REST API allows users to:

- Create an account
- Log in securely
- View their own profile
- Create blog posts
- View public blog posts
- Update their own blog posts
- Delete their own blog posts

Authentication will use JWT access tokens. Passwords will be hashed with `bcryptjs`. MySQL will store users and blogs. Postman will be used for manual API testing.

A frontend, deployment, comments, likes, and refresh tokens are outside version 1 of the product.

4. Problem Statement

A blog platform requires a secure backend that identifies users, stores credentials safely, and prevents one user from modifying another user's content.

The API must ensure that:

- Plain-text passwords are never stored.
- Protected operations require a valid JWT.
- The authenticated user's UUID is trusted instead of a user ID supplied in the request body.
- A user can update or delete only their own blog posts.
- Invalid data is rejected consistently before reaching a controller.
- SQL injection is prevented through parameterized queries.
- The codebase remains understandable through a class-based modular structure.

5. Project Objectives

The project must demonstrate practical understanding of:

- JavaScript classes and object-oriented design
- Constructor-based dependency injection
- Node.js and Express.js
- REST API design
- Express routing and middleware chains
- Middleware-first validation and authorization
- Thin controllers
- Service and database-service separation
- MySQL and relational design
- UUID identifiers
- JWT authentication
- Password hashing
- Resource ownership
- Centralized error handling
- Constants management
- SQL injection prevention

- Postman testing
- Git branching and conventional commits
- Automated formatting with Prettier

6. Technology Stack

6.1 Core technologies

- JavaScript
- Node.js
- Express.js
- MySQL
- Git
- GitHub
- Postman

6.2 Runtime dependencies

```
npm install express mysql2 jsonwebtoken bcryptjs dotenv cors
```

UUIDs will be generated with the built-in Node.js API:

```
const { randomUUID } = require('node:crypto');
```

No additional UUID package is required unless the project environment has a specific reason to use one.

6.3 Development dependencies

```
npm install --save-dev nodemon prettier
```

6.4 Package responsibilities

- **express**: server, routes, middleware integration, HTTP responses
- **mysql2**: MySQL connection pool and parameterized queries
- **jsonwebtoken**: JWT generation and verification
- **bcryptjs**: password hashing and comparison
- **dotenv**: environment-variable loading
- **cors**: cross-origin access configuration
- **nodemon**: development server restart
- **prettier**: consistent code and document formatting

7. Target Users

7.1 Guest user

A guest can:

- Register an account
- Log in
- View all blog posts
- View one blog post

A guest cannot create, update, or delete a blog post.

7.2 Registered user

A registered user can:

- Log in
- View their profile

- Create blog posts
- View public blog posts
- Update their own blog posts
- Delete their own blog posts

A registered user cannot manage another user's blog posts.

8. Product Scope

8.1 In scope

- User registration
- User login
- UUID user IDs
- Password hashing
- JWT generation and verification
- Current-user profile endpoint
- Blog CRUD
- User-to-blog relationship
- Middleware validation
- Middleware authentication
- Middleware resource existence checks
- Middleware ownership checks
- Class-based architecture
- Centralized error handling
- MySQL integration
- Constants organization
- Prettier configuration
- Postman collection and environment
- Git feature branches and proper commit history
- README documentation

8.2 Out of scope

- Frontend application
- Social login
- Password reset email
- Email verification
- Refresh tokens
- Logout-token invalidation
- Admin dashboard
- User roles
- Comments and likes
- Categories and tags
- Image upload
- Notifications
- Rich text editor
- Deployment

9. Core Architecture Principles

9.1 Class-based modular architecture

Every main architectural component must be a class. A typical dependency flow is:

Route instance

- > middleware class methods
- > controller class method

- > main service class method
- > database service class method
- > query constants
- > MySQL

9.2 Middleware-first request processing

Middleware is responsible for preparing a valid request. By the time a controller runs, the request must already be authenticated, validated, authorized, normalized, and enriched with required records.

9.3 Thin controllers

A controller should be predictable and very small. It should not decide what is valid or allowed.

Recommended controller shape:

```
class BlogController {
  constructor(blogService) {
    this.blogService = blogService;
    this.updateBlog = this.updateBlog.bind(this);
  }

  async updateBlog(req, res) {
    const result = await this.blogService.updateBlog({
      blog: req.blog,
      changes: req.validatedBody,
    });

    return res.status(HTTP_STATUS.OK).json(result);
  }
}
```

The controller above has no validation, ownership check, existence check, or data normalization.

9.4 Services execute use cases

Services may:

- Coordinate database writes and reads needed to complete a use case
- Generate UUIDs for new users
- Hash a password after middleware has validated it
- Generate a JWT after credential middleware has authenticated the login
- Build standard success-result objects
- Call filter services to create safe output

Services must not repeat request checks already completed by middleware.

9.5 Database services only access the database

Database services may:

- Execute parameterized queries
- Supply query parameters
- Return rows, insert results, update results, and delete results

Database services must not:

- Read Express request or response objects
- Send HTTP responses
- Perform permission decisions
- Contain controller logic

9.6 Query files contain SQL only

SQL statements must be stored in query files as constants. Queries must use placeholders and must never concatenate user input.

9.7 Constants eliminate magic values

Repeated messages, field limits, status codes, bcrypt salt rounds, and similar fixed values must come from a constants file.

10. Functional Requirements

10.1 User registration

Route

POST /api/auth/register

Request body

```
{
  "name": "Momitul",
  "email": "momitul@example.com",
  "password": "SecurePass123"
}
```

Middleware chain

1. Validate content type and body presence.
2. Validate required fields.
3. Validate email format.
4. Validate password length.
5. Normalize name and email.
6. Check that the email is not already registered.
7. Attach sanitized data to `req.validatedBody`.

Service processing

1. Generate a UUID using `randomUUID()`.
2. Hash the validated password using the configured `SALT_ROUNDS`.
3. Insert the user through the auth database service.
4. Return a filtered user object without a password.

Expected response

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Momitul",
    "email": "momitul@example.com"
  }
}
```

Status: 201 Created

10.2 User login

Route

POST /api/auth/login

Middleware chain

1. Validate email and password presence.
2. Normalize the email.
3. Load the user by email.
4. Compare the submitted password with the stored hash.
5. Reject invalid credentials with one generic message.
6. Attach the authenticated user to `req.authenticatedUser`.

Controller and service

The controller sends `req.authenticatedUser` to the auth service. The service generates the token and returns safe user information.

Expected response

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "user": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Momitul",
      "email": "momitul@example.com"
    },
    "token": "jwt-token"
  }
}
```

Status: 200 OK

10.3 Get current user

Route

GET /api/auth/me

Middleware chain

1. Read the **Authorization** header.
2. Validate the Bearer format.
3. Verify the JWT.
4. Validate the decoded user UUID.
5. Load the current user.
6. Reject a token whose user no longer exists.
7. Attach the safe current user to `req.currentUser`.

The controller returns `req.currentUser` through the service or response wrapper. The password must never be returned.

10.4 Create a blog post

Route

POST /api/blogs

Middleware chain

1. Authenticate the JWT.
2. Load the current user.
3. Validate title and content.
4. Enforce title length.
5. Trim title and content.
6. Attach sanitized data to `req.validatedBody`.

The service inserts the blog with `req.currentUser.id` as `user_id`. A user ID from the request body must be ignored or rejected.

10.5 Get all blog posts

Route

GET `/api/blogs`

Guests and authenticated users may access the route. The response must contain safe author data and must never contain password fields.

10.6 Get one blog post

Route

GET `/api/blogs/:id`

Middleware chain

1. Validate that `id` is a positive integer.
2. Load the blog.
3. Return 404 Not Found when missing.
4. Attach the blog to `req.blog`.

10.7 Update a blog post

Route

PATCH `/api/blogs/:id`

Middleware chain

1. Authenticate the JWT.
2. Validate the blog ID.
3. Load the blog.
4. Verify that the current user owns the blog.
5. Validate that at least one allowed field is supplied.
6. Allow only `title` and `content`.
7. Reject `user_id`, `id`, timestamps, and unknown fields.
8. Validate and normalize supplied values.
9. Attach the blog to `req.blog` and changes to `req.validatedBody`.

The controller must not contain any ownership or existence logic.

10.8 Delete a blog post

Route

DELETE `/api/blogs/:id`

Middleware chain

1. Authenticate the JWT.
2. Validate the blog ID.
3. Load the blog.
4. Verify ownership.
5. Attach the authorized blog to `req.blog`.

Expected response:

```
{
  "success": true,
  "message": "Blog deleted successfully"
}
```

11. API Endpoints

11.1 Health

Method	Route	Access	Purpose
GET	/health	Public	Verify server operation

11.2 Authentication

Method	Route	Access	Purpose
POST	/api/auth/register	Public	Register a user
POST	/api/auth/login	Public	Authenticate a user
GET	/api/auth/me	Private	Get current user

11.3 Blogs

Method	Route	Access	Purpose
POST	/api/blogs	Private	Create a blog
GET	/api/blogs	Public	Get all blogs
GET	/api/blogs/:id	Public	Get one blog
PATCH	/api/blogs/:id	Owner only	Update a blog
DELETE	/api/blogs/:id	Owner only	Delete a blog

11.4 Optional user module after core completion

Method	Route	Access	Purpose
GET	/api/users/:id	Defined by policy	Get a user
PATCH	/api/users/:id	Same user only	Update a user
DELETE	/api/users/:id	Same user only	Delete a user

12. Database Requirements

12.1 Database

`blog_database`

12.2 Users table with UUID primary key

```
CREATE TABLE users (
  id CHAR(36) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP
);
```

The application must generate the UUID before insertion:

```
const userId = randomUUID();
```

12.3 Blogs table

Blog IDs may remain auto-incrementing integers unless the instructor later requires UUIDs for blogs.

```
CREATE TABLE blogs (
  id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  user_id CHAR(36) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_blogs_user
  FOREIGN KEY (user_id)
  REFERENCES users(id)
  ON DELETE CASCADE
);
```

12.4 Relationship

- One user can create many blog posts.
- One blog post belongs to one user.
- `blogs.user_id` references `users.id`.
- Deleting a user deletes that user's blogs through `ON DELETE CASCADE`.

13. Required Project Structure

```
jwt-blog-api/
|
|-- src/
|   |-- config/
|   |   |-- env.config.js
|   |
|   |-- constant/
|   |   |-- app.constant.js
|   |   |-- http-status.constant.js
|   |   |-- error-message.constant.js
|   |
|   |-- db/
|   |   |-- mysql/
|   |       |-- mysql.connection.js
|   |       |-- mysql.init.js
|   |
```

15

```

| | | | -- user-ownership.middleware.js
| | | | -- query/
| | | | -- user.query.js
| | | | -- service/
| | | | -- user.service.js
| | | | -- user-db.service.js
| | | | -- user-filter.service.js
| | | | -- user.routes.js
| |
| | -- app.js
| | -- index.js
|
|-- database/
| | -- schema.sql
|
|-- postman/
| | -- JWT-Blog-API.postman_collection.json
| | -- JWT-Blog-API.postman_environment.json
|
|-- .env
|-- .env.example
|-- .gitignore
|-- .prettierrc.json
|-- package.json
|-- package-lock.json
|-- README.md

```

14. Required Request Flow

14.1 Generic flow

Client/Postman

- > Express route
- > ordered middleware checks
- > controller class
- > service class
- > database service class
- > SQL query constant
- > MySQL
- > filtered response
- > client

14.2 Update-blog flow

PATCH /api/blogs/:id

- > JwtAuthMiddleware.authenticate
- > CurrentUserMiddleware.load
- > BlogValidationMiddleware.validateId
- > BlogExistenceMiddleware.load
- > BlogOwnershipMiddleware.authorize
- > BlogValidationMiddleware.validateUpdateBody
- > BlogController.updateBlog
- > BlogService.updateBlog
- > BlogDbService.updateById
- > BLOG_QUERIES.UPDATE_BY_ID
- > MySQL

Every check occurs before `BlogController.updateBlog`.

15. Class-Based Coding Standard

15.1 Controller class

```
class AuthController {
  constructor(authService) {
    this.authService = authService;
    this.register = this.register.bind(this);
    this.login = this.login.bind(this);
    this.me = this.me.bind(this);
  }

  async register(req, res) {
    const result = await this.authService.register(req.validatedBody);
    return res.status(HTTP_STATUS.CREATED).json(result);
  }

  async login(req, res) {
    const result = await this.authService.login(req.authenticatedUser);
    return res.status(HTTP_STATUS.OK).json(result);
  }

  async me(req, res) {
    const result = await this.authService.getCurrentUser(req.currentUser);
    return res.status(HTTP_STATUS.OK).json(result);
  }
}
```

No checking logic is permitted in these methods.

15.2 Middleware class

```
class BlogOwnershipMiddleware {
  authorize(req, res, next) {
    if (req.blog.userId !== req.currentUser.id) {
      throw new AppError(
        HTTP_STATUS.FORBIDDEN,
        BLOG_MESSAGES.NOT_OWNER
      );
    }

    return next();
  }
}
```

This ownership decision belongs in middleware, not in the controller.

15.3 Service class

```
class AuthService {
  constructor(authDbService, passwordService, tokenService, filterService) {
    this.authDbService = authDbService;
    this.passwordService = passwordService;
    this.tokenService = tokenService;
    this.filterService = filterService;
  }
}
```

```

async register(data) {
  const id = randomUUID();
  const hashedPassword = await this.passwordService.hash(data.password);

  const user = await this.authDbService.create({
    id,
    name: data.name,
    email: data.email,
    password: hashedPassword,
  });

  return {
    success: true,
    message: AUTH_MESSAGES.REGISTER_SUCCESS,
    data: this.filterService.toSafeUser(user),
  };
}
}

```

The service creates the UUID and performs the use case. Duplicate-email checking has already happened in middle-ware.

15.4 Database service class

```

class AuthDbService {
  constructor(pool) {
    this.pool = pool;
  }

  async create(user) {
    await this.pool.execute(AUTH_QUERIES.CREATE_USER, [
      user.id,
      user.name,
      user.email,
      user.password,
    ]);

    const [rows] = await this.pool.execute(AUTH_QUERIES.FIND_BY_ID, [
      user.id,
    ]);

    return rows[0];
  }
}

```

16. File Responsibilities

16.1 Routes

Routes must:

- Define method and path
- Attach middleware in the correct order
- Call a bound controller method

Routes must not contain validation, SQL, ownership logic, or response construction beyond route registration.

16.2 Middleware

Middleware must:

- Perform all request checks
- Stop invalid requests before controllers
- Throw or forward consistent application errors
- Attach validated or loaded data to `req`
- Avoid sending successful business responses

16.3 Controllers

Controllers must:

- Read prepared request properties
- Call one service method
- Return the service result with the correct status

Controllers must remain free of checking and business logic.

16.4 Main services

Services must:

- Execute the use case
- Coordinate helper, password, token, filter, and database services
- Avoid Express-specific request and response objects

16.5 Database services

Database services must:

- Execute parameterized queries
- Return database results
- Avoid HTTP and permission concerns

16.6 Query files

Query files must:

- Export SQL strings
- Use placeholders
- Avoid user-input concatenation

16.7 Filter services

Filter services must:

- Remove password hashes
- Convert database field names when required
- Return safe API objects

17. Constants Requirements

17.1 Auth constants example

```
const AUTH_CONSTANTS = Object.freeze({
  SALT_ROUNDS: 10,
  PASSWORD_MIN_LENGTH: 8,
  TOKEN_PREFIX: 'Bearer',
});
```

```
module.exports = AUTH_CONSTANTS;
```

SALT_ROUNDS must not be written directly inside `bcrypt.hash()`.

Correct:

```
bcrypt.hash(password, AUTH_CONSTANTS.SALT_ROUNDS);
```

Incorrect:

```
bcrypt.hash(password, 10);
```

17.2 Blog constants example

```
const BLOG_CONSTANTS = Object.freeze({
  TITLE_MAX_LENGTH: 255,
  UPDATEABLE_FIELDS: ['title', 'content'],
});
```

17.3 Constants placement rules

Value	Required location
SALT_ROUNDS	<code>modules/auth/constant/auth.constant.js</code>
Password minimum length	<code>modules/auth/constant/auth.constant.js</code>
Blog title maximum	<code>modules/blog/constant/blog.constant.js</code>
Blog updateable fields	<code>modules/blog/constant/blog.constant.js</code>
HTTP status codes	<code>src/constant/http-status.constant.js</code>
Auth messages	Auth module constants
Blog messages	Blog module constants
Port, DB settings, JWT secret	<code>.env</code> and <code>env</code> config

18. Authentication and Security Requirements

18.1 Passwords

- Never store plain-text passwords.
- Never return password hashes.
- Hash passwords using `bcryptjs` and `AUTH_CONSTANTS.SALT_ROUNDS`.
- Compare login passwords only in credential-checking middleware or a password service called by that middleware.

18.2 JWT payload

The token should contain only necessary data:

```
{
  "userId": "550e8400-e29b-41d4-a716-446655440000",
  "email": "momitul@example.com"
}
```

The token must not contain a password or password hash.

18.3 JWT expiry

The expiry must come from environment configuration:

```
JWT_EXPIRES_IN=1d
```

18.4 Ownership

- Ownership checks must run in middleware.
- The API must compare `req.blog.userId` with `req.currentUser.id`.
- A `user_id` supplied by the client must never determine ownership.

18.5 SQL injection prevention

All SQL must use parameter placeholders:

```
SELECT * FROM users WHERE email = ?;
```

User input must never be concatenated into SQL strings.

19. Validation and Middleware Matrix

Operation	Required middleware checks
Register	Body, name, email, password, normalization, duplicate email
Login	Body, email, password, normalization, user lookup, password comparison
Current user	Bearer header, token verification, UUID validation, user existence
Create blog	Authentication, current user, body, title, content, title length
Get blog	Blog ID format, blog existence
Update blog	Authentication, blog ID, existence, ownership, non-empty body, allow-list, value validation
Delete blog	Authentication, blog ID, existence, ownership
Get/update user	UUID parameter, user existence, authorization as applicable

20. Error Handling Requirements

The API must return a consistent error structure:

```
{
  "success": false,
  "message": "Blog not found"
}
```

20.1 Required status codes

Status	Use
200 OK	Successful read, update, or login
201 Created	Successful resource creation
400 Bad Request	Invalid request input
401 Unauthorized	Missing or invalid authentication
403 Forbidden	Authenticated but not permitted
404 Not Found	Route or resource missing
409 Conflict	Duplicate email
500 Internal Server Error	Unexpected server failure

20.2 Error middleware order

```
app.use(notFoundMiddleware.handle);
app.use(errorMiddleware.handle);
```

These must be registered after all API routes.

21. Environment Configuration

21.1 .env

```
NODE_ENV=development
PORT=5000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=blog_database
JWT_SECRET=your_private_jwt_secret
JWT_EXPIRES_IN=1d
```

21.2 .env.example

```
NODE_ENV=development
PORT=5000
DB_HOST=
DB_PORT=3306
DB_USER=
DB_PASSWORD=
DB_NAME=blog_database
JWT_SECRET=
JWT_EXPIRES_IN=1d
```

Secrets must not be committed.

22. Prettier and Formatting Requirements

22.1 .prettierrc.json

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100
}
```

22.2 .prettierignore

```
node_modules/
coverage/
.env
package-lock.json
```

22.3 Package scripts

```
{
  "scripts": {
```

```
    "dev": "nodemon src/index.js",
    "start": "node src/index.js",
    "format": "prettier --write .",
    "format:check": "prettier --check ."
  }
}
```

22.4 Formatting gate

Before every commit intended for review:

```
npm run format
npm run format:check
```

A branch must not be merged when `format:check` fails.

23. Postman Testing Plan

23.1 Collection structure

```
JWT Blog API
|-- Health
|   |-- Check server
|-- Auth
|   |-- Register user
|   |-- Login user
|   |-- Get current user
|-- Blogs
|   |-- Create blog
|   |-- Get all blogs
|   |-- Get blog by ID
|   |-- Update blog
|   |-- Delete blog
|-- Users
    |-- Get user
    |-- Update user
    |-- Delete user
```

23.2 Environment variables

- baseUrl
- token
- userId
- blogId

Example:

```
baseUrl=http://localhost:5000/api
```

23.3 Login test script

```
const response = pm.response.json();

if (response.data && response.data.token) {
  pm.environment.set('token', response.data.token);
}
```

23.4 Required tests

Authentication

- Valid registration
- Missing name
- Missing email
- Invalid email
- Short password
- Duplicate email
- Correct login
- Incorrect password
- Unregistered email
- Profile without token
- Profile with invalid token
- Profile with valid token

Blogs

- Create with valid token
- Create without token
- Create with invalid token
- Create without title
- Create without content
- Get all
- Get existing blog
- Get nonexistent blog
- Update own blog
- Update another user's blog
- Update with empty body
- Update with forbidden field
- Delete own blog
- Delete another user's blog
- Delete nonexistent blog

UUID-specific tests

- User response contains a valid UUID
- JWT contains the UUID user ID
- Invalid user UUID route parameter returns 400
- Blog foreign key stores the same user UUID

24. Git and GitHub Workflow

24.1 Required branches

- **main**: stable, review-ready code only
- **develop**: integration branch for completed features
- **feature/<scope>**: new feature development
- **fix/<scope>**: bug fixes
- **refactor/<scope>**: structural improvements without new behavior
- **docs/<scope>**: documentation-only changes
- **chore/<scope>**: tooling and maintenance

Examples:

```
feature/class-based-auth  
feature/uuid-user-schema  
feature/blog-middleware
```



```
chore/prettier-setup
docs/api-readme
```

24.2 Required workflow

```
git checkout develop
git pull origin develop
git checkout -b feature/class-based-auth
```

After development:

```
npm run format
npm run format:check
git status
git add <specific-files>
git commit -m "feat(auth): add class-based registration flow"
git push -u origin feature/class-based-auth
```

Open a pull request into develop. Merge develop into main only after the milestone is stable.

24.3 Commit rules

- One commit should represent one understandable change.
- Do not use vague messages such as `update`, `changes`, `done`, or `final`.
- Do not combine unrelated modules in one commit.
- Do not commit `.env` or `node_modules`.
- Run Prettier before committing.
- Prefer staging specific files over blindly staging unrelated changes.

24.4 Conventional commit examples

```
chore: initialize Node.js project
chore(format): configure Prettier
refactor(core): convert middleware to classes
feat(db): add UUID-based users schema
feat(auth): add registration validation middleware
feat(auth): add duplicate-email middleware
feat(auth): add password hashing service
feat(auth): add login credential middleware
feat(auth): add JWT authentication middleware
feat(blog): add blog validation middleware
feat(blog): add blog existence middleware
feat(blog): add blog ownership middleware
feat(blog): add class-based blog CRUD services
fix(auth): return generic invalid-credentials message
docs: add setup and API usage instructions
test(postman): add ownership and UUID test cases
```

24.5 Suggested feature order

1. chore/project-setup
2. chore/prettier-setup
3. feature/mysql-connection
4. feature/uuid-user-schema
5. feature/class-based-auth-registration
6. feature/class-based-auth-login
7. feature/jwt-middleware
8. feature/blog-read-create
9. feature/blog-update-delete

- 10. feature/blog-ownership-middleware
- 11. feature/centralized-errors
- 12. test/postman-collection
- 13. docs/readme

25. Development Milestones

Milestone 1: Project foundation

- Initialize npm
- Install dependencies
- Add modular folders
- Add class instances and dependency wiring
- Configure environment variables
- Configure Prettier
- Add Express server and health route
- Initialize Git branches

Milestone 2: MySQL and schema

- Create database
- Create UUID users table
- Create blogs table with UUID foreign key
- Create connection pool class/configuration
- Verify connection

Milestone 3: Registration

- Registration validation middleware class
- Email normalization
- Duplicate-email middleware class
- UUID generation
- Password service with constant salt rounds
- Auth database service class
- Thin auth controller method

Milestone 4: Login and JWT

- Login validation middleware
- Credential middleware
- Token service class
- Login controller and service
- Generic invalid-credential errors

Milestone 5: Protected profile

- JWT middleware class
- Current-user existence middleware
- Safe profile response

Milestone 6: Blog creation and reading

- Blog validation middleware
- Blog controller, service, database service classes
- Create and read endpoints
- Safe author response

Milestone 7: Blog update and deletion

- Blog-ID middleware
- Blog-existence middleware
- Blog-ownership middleware
- Update allow-list middleware
- Thin update/delete controllers

Milestone 8: Errors and consistency

- Application error class
- Not-found middleware class
- Centralized error middleware class
- Consistent messages and status constants

Milestone 9: Testing

- Postman collection
- Environment variables
- Automatic token storage
- Success, failure, ownership, and UUID tests

Milestone 10: Documentation and release

- Complete README
- Add schema and endpoint documentation
- Export Postman files
- Verify branch and commit history
- Run formatting check
- Merge stable `develop` into `main`

26. Definition of Done

The project is complete only when all of the following are true:

- The server starts without errors.
- MySQL connection succeeds.
- The architecture is class-based.
- Controllers contain no validation, authorization, existence, ownership, or business checks.
- Every request check is implemented in middleware.
- User IDs are UUIDs.
- User UUIDs are stored as the users primary key.
- Blog foreign keys reference user UUIDs.
- Registration works.
- Duplicate email is rejected before the controller.
- Passwords are hashed using `SALT_ROUNDS` from an auth constants file.
- Login credential checking happens before the controller.
- Login returns a valid expiring JWT.
- JWT middleware protects private routes.
- Authenticated users can create blogs.
- Anyone can read blogs.
- Owners can update and delete their blogs.
- Non-owners receive **403 Forbidden**.
- Missing resources receive **404 Not Found**.
- Invalid input receives a consistent error response.
- No password or hash is returned.
- All queries use placeholders.
- `.env` and `node_modules` are ignored.

- Prettier configuration is committed.
- `npm run format:check` passes.
- Work was completed through proper branches.
- Commit history is clear and meaningful.
- Postman tests pass.
- README setup instructions are complete.

27. Acceptance Criteria

27.1 Architecture acceptance

- Given any controller file, it contains no request validation, ownership check, resource existence check, token verification, or credential comparison.
- Given a protected route, its authentication and authorization middleware execute before the controller.
- Given the source tree, controllers, services, database services, and module middleware are class-based.

27.2 Registration acceptance

- Given valid data, a UUID user is created.
- Given a duplicate email, middleware returns **409 Conflict** and the controller is not called.
- The database contains a password hash, not a plain password.
- The response excludes the password.

27.3 Login acceptance

- Given correct credentials, credential middleware attaches the authenticated user and the service returns a JWT.
- Given incorrect credentials, the API returns **401 Unauthorized** with a generic message.
- The token contains the UUID user ID and an expiry.

27.4 Blog acceptance

- An authenticated user can create a blog linked to their UUID.
- An unauthenticated create request returns **401 Unauthorized** before the controller.
- A valid blog can be read publicly.
- A missing blog returns **404 Not Found** from middleware.
- An owner can update and delete a blog.
- A non-owner receives **403 Forbidden** from middleware.
- An empty or forbidden update is rejected before the controller.

27.5 Git and formatting acceptance

- New features exist on named feature branches.
- Commits use meaningful conventional messages.
- `.prettierrc.json` and `.prettierignore` exist.
- `npm run format:check` exits successfully.
- The repository does not track `.env` or `node_modules`.

28. Codex Execution Instructions

Codex must implement the project according to this document with the following working rules:

1. Read the whole document before changing code.
2. Preserve existing working behavior unless a requirement in this document changes it.
3. Use classes for all controllers, services, database services, helper/filter services, and module middleware.
4. Do not place any request checking in controllers.
5. Build route middleware chains so checks complete before controllers.
6. Generate user UUIDs with `crypto.randomUUID()`.

7. Store fixed values in the correct constants folder; never hard-code `SALT_ROUNDS` in a hashing call.
8. Use parameterized SQL only.
9. Keep controllers thin and predictable.
10. Add or update Postman cases for every endpoint and failure path.
11. Run Prettier after each logical implementation step.
12. Work in a feature branch and create small conventional commits.
13. Do not commit secrets, `.env`, `node_modules`, generated logs, or unrelated files.
14. Before declaring a task complete, run the server, test the route, run `npm run format:check`, and inspect `git status`.
15. When requirements conflict, the Mandatory Instructor Requirements section has highest priority.

29. Future Enhancements

After the core project is complete, possible additions include:

- Admin and user roles
- Draft and published states
- Categories and tags
- Comments and likes
- Pagination
- Search and sorting
- Image uploads
- Password reset
- Email verification
- Refresh tokens
- Rate limiting
- Automated unit and integration tests
- Swagger/OpenAPI documentation
- Frontend application
- Cloud deployment

30. Final Product Summary

The final product will be a secure, class-based REST API built with JavaScript, Node.js, Express.js, and MySQL.

It will demonstrate:

- UUID-based user identity
- JWT authentication
- Password hashing
- Middleware-first checking
- Logic-free controllers
- Class-based services and database access
- Blog CRUD and ownership authorization
- Constants organized by responsibility
- Centralized errors
- Parameterized SQL
- Postman testing
- Prettier-enforced formatting
- Proper Git branching and commit discipline

The most important architectural rule is that controllers receive a request that has already been checked and prepared by middleware. Controllers only call services and return responses.